



iOS App Store Deployment Guide

From your Mac to the App Store, written for a first-time Xcode user

Mobile edition · Capacitor shell · Version 1.0

How the Mobile App Works

The iOS app is a native shell built with Capacitor that loads the production platform at policytoaction.beauhawked.com inside an embedded, full-screen browser view. Because every account's data lives on the server, a user who signs in on the iPhone app sees exactly the same policies, handbooks, conversations, and pinned answers as on the web, with seamless handoff in both directions. There is no separate mobile database and nothing to synchronize.

Already prepared for you: the Capacitor configuration, the app identity (`com.scarletfire.policytoaction`), the stub offline shell, the icon and splash source assets built from the Policy to Action logo, and safe-area handling so the interface clears the iPhone notch and home indicator.

Phase 1: One-Time Prerequisites

- 1 Enroll in the Apple Developer Program** at developer.apple.com/programs (99 dollars per year). Enroll as an organization under Scarlet Fire LLC if you have a D-U-N-S number, or as an individual otherwise; individual enrollment is faster and can be migrated later. Approval can take a day or two.
- 2 Install Xcode** from the Mac App Store (free, large download). Open it once after installing and accept the license, letting it install its additional components.
- 3 Install CocoaPods**, the dependency manager Capacitor uses for iOS. In Terminal: `sudo gem install cocoapods` and enter your Mac password when asked. If that errors, install Homebrew from brew.sh and run `brew install cocoapods` instead.

- 4 **Sign in to Xcode with your Apple ID:** Xcode menu, Settings, Accounts, the plus button, Apple Account. Use the same Apple ID as your developer enrollment.

Phase 2: Generate the iOS Project

In Terminal, run these commands one at a time:

```
cd ~/Vault/Projects/Dissertation-IU/Policy-to-Action
npm install
npx cap add ios
npm install -D @capacitor/assets
npx capacitor-assets generate --ios
npx cap sync ios
```

What each does: install the new Capacitor packages; create the native iOS project in an ios folder; install the asset generator; stamp the app icon and splash screens from the prepared logo assets; and synchronize the configuration into the iOS project. If `cap sync` completes without red errors, the project is ready.

Phase 3: First Build in Xcode

- 1 **Open the project:** `npx cap open ios` launches Xcode with the workspace loaded.
- 2 **Set your signing team.** In the left sidebar click the blue **App** project icon at the top, select the **App** target in the center pane, open the **Signing & Capabilities** tab, tick **Automatically manage signing**, and choose your team in the Team dropdown. The bundle identifier `com.scarletfire.policytoaction` is already set.
- 3 **Run it in the simulator.** In the device selector at the top center of Xcode, pick an iPhone simulator, then press the Play button. The simulator boots, the splash shows, and the live platform loads. Sign in and confirm your conversations and pins appear, proving the cross-platform consistency requirement.

Phase 4: TestFlight and the App Store

- 1 **Create the App Store record.** At appstoreconnect.apple.com choose Apps, the plus button, New App: platform iOS, name Policy to Action, primary language English, bundle ID `com.scarletfire.policytoaction` (it appears after your first upload or can be registered under Identifiers at developer.apple.com), SKU `policy-to-action`.
- 2 **Archive and upload.** In Xcode set the device selector to **Any iOS Device (arm64)**, then menu Product, Archive. When the Organizer window opens, select the archive, click **Distribute App**, choose **App Store Connect**, and accept the defaults through to Upload.

- 3 **TestFlight first.** In App Store Connect, TestFlight tab, the build appears after processing (roughly 15 to 60 minutes). Add yourself as an internal tester, install TestFlight on your iPhone, and use the real app for a day. This is also your dissertation-grade verification of the handoff requirement on hardware.
- 4 **Complete the listing.** App Store tab: description, keywords, support URL (use the Help page), marketing URL, and screenshots. Take screenshots in the iPhone simulator (File, then Save Screen) on a 6.7 inch device such as iPhone 15 Pro Max; the ones from your dev workflow translate well: sign-in, question screen, cited answer, evidence panel.
- 5 **App Privacy labels.** Declare collection of: Name, Email Address, and User Content (the scenarios and conversations), all linked to identity, none used for tracking, no third-party advertising. This mirrors the Trust and Privacy page exactly.
- 6 **Submit for review with strong notes.** Provide a demo account (create a dedicated reviewer account with a loaded sample policy dataset) and review notes explaining the app: professional decision-support software for school district administrators, with accounts and district data provisioned through the organization. Apple's Guideline 4.2 discourages thin website wrappers, so the notes matter: emphasize the account-based professional service, the structured policy retrieval, and that the app is the mobile access point to an organizational platform. If the reviewer raises 4.2, the planned remediation is adding push notifications, which we can implement on request.

Common First-Timer Snags

Symptom	Resolution
Signing error: no team	Xcode Settings, Accounts: confirm your Apple ID is added and your developer enrollment is approved.
pod: command not found	CocoaPods is not installed; revisit Phase 1 step 3, then rerun <code>npx cap sync ios</code> .
Archive is greyed out	The device selector must read Any iOS Device (arm64), not a simulator.
Build appears in App Store Connect but not TestFlight	Processing takes up to an hour; also check email for a missing-compliance notice, and answer the encryption question (the app uses only standard HTTPS encryption, which qualifies for the exemption).
White screen on launch	The device has no network path to <code>policytoaction.beauhawked.com</code> ; the shell requires connectivity.

Android, When Ready

The Android path mirrors this one with Android Studio instead of Xcode: `npx cap add android`, asset generation with `--android`, a one-time 25 dollar Google Play Console registration, and an AAB build uploaded through the Play Console. The same shell, configuration, and assets serve both platforms.

Policy to Action iOS Deployment Guide, Version 1.0. The mobile app is a native shell over the production platform; all user data remains server side, giving identical state across web, iOS, and Android.